Week 5: Ethical Hacking & Exploiting Vulnerabilities

>Github

1. Introduction

The objective of this task was to perform ethical hacking on a test web application to identify potential vulnerabilities and improve overall security. The assessment focused on SQL Injection (SQLi) and Cross-Site Request Forgery (CSRF) using industry-standard tools.

2. Tools & Environment

- **Operating System:** Kali Linux
- **Backend Application:** Node.js (Express)
- **Testing Tools:**
  - Burp Suite
  - SQLMap
  - Thunder Client

3. Reconnaissance

Target

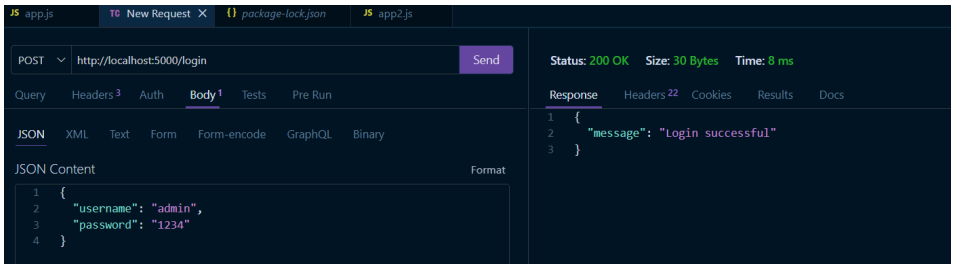
http://localhost:5000

> Identified Endpoints

- GET /
- GET /api/data
- POST /login

> Security Observed

- API Key authentication
- Rate limiting
- Security headers



> Show:

- `/login` request working
- Header + body

4. SQL Injection Testing

Command Used

```
sqlmap -u "http://localhost:5000/login" \

--data='{"username":"admin","password":"1234"}' \

--headers="Content-Type: application/json, x-api-key:
mysecurekey123" \

--method=POST --dbs
```

```
┌──(kali㉿kali)-[~/Downloads/Intern]
└─$ sqlmap -u "http://localhost:5000/login" \
--data='{"username":"admin","password":"1234"}' \
--headers="Content-Type: application/json, x-api-key: mysecurekey123" \
--method=POST --dbs
        ___
       __H__
 ___ ___[)]_____ ___ ___  {1.9.3#stable}
|_ -| . [)]     | .'| . |
|___|_  [)]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicab
le local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 05:47:21 /2026-03-24/

JSON data found in POST body. Do you want to process it? [Y/n/q] Y
[05:47:30] [INFO] testing connection to the target URL
[05:47:30] [WARNING] the web server responded with an HTTP error code (403) which could interfere with the results of the tests
[05:47:30] [INFO] checking if the target is protected by some kind of WAF/IPS
[05:47:30] [INFO] testing if the target URL content is stable
[05:47:31] [INFO] target URL content is stable
[05:47:31] [INFO] testing if (custom) POST parameter 'JSON username' is dynamic
[05:47:31] [WARNING] (custom) POST parameter 'JSON username' does not appear to be dynamic
[05:47:31] [WARNING] heuristic (basic) test shows that (custom) POST parameter 'JSON username' might not be injectable
[05:47:31] [INFO] testing for SQL injection on (custom) POST parameter 'JSON username'
[05:47:31] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[05:47:31] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[05:47:31] [INFO] testing 'MySQL ≥ 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[05:47:31] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[05:47:31] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[05:47:31] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[05:47:31] [INFO] testing 'Generic inline queries'
[05:47:31] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[05:47:31] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[05:47:31] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[05:47:31] [INFO] testing 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)'
[05:47:31] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[05:47:31] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[05:47:31] [INFO] testing 'Oracle AND time-based blind'
it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to reduce the number of reque
```

```
sts? [Y/n] Y
[05:47:35] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[05:47:35] [WARNING] (custom) POST parameter 'JSON username' does not seem to be injectable
[05:47:35] [INFO] testing if (custom) POST parameter 'JSON password' is dynamic
[05:47:35] [INFO] (custom) POST parameter 'JSON password' appears to be dynamic
[05:47:35] [WARNING] heuristic (basic) test shows that (custom) POST parameter 'JSON password' might not be injectable
[05:47:35] [INFO] testing for SQL injection on (custom) POST parameter 'JSON password'
[05:47:35] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[05:47:35] [INFO] testing 'Boolean-based blind - Parameter replace (original value)'
[05:47:35] [INFO] testing 'MySQL ≥ 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[05:47:35] [INFO] testing 'PostgreSQL AND error-based - WHERE or HAVING clause'
[05:47:35] [INFO] testing 'Microsoft SQL Server/Sybase AND error-based - WHERE or HAVING clause (IN)'
[05:47:35] [INFO] testing 'Oracle AND error-based - WHERE or HAVING clause (XMLType)'
[05:47:35] [INFO] testing 'Generic inline queries'
[05:47:35] [INFO] testing 'PostgreSQL > 8.1 stacked queries (comment)'
[05:47:35] [INFO] testing 'Microsoft SQL Server/Sybase stacked queries (comment)'
[05:47:35] [INFO] testing 'Oracle stacked queries (DBMS_PIPE.RECEIVE_MESSAGE - comment)'
[05:47:35] [INFO] testing 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)'
[05:47:35] [INFO] testing 'PostgreSQL > 8.1 AND time-based blind'
[05:47:35] [INFO] testing 'Microsoft SQL Server/Sybase time-based blind (IF)'
[05:47:35] [INFO] testing 'Oracle AND time-based blind'
[05:47:35] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
[05:47:35] [WARNING] (custom) POST parameter 'JSON password' does not seem to be injectable
[05:47:35] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options if you wish to perform more
 tests. If you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space
2comment') and/or switch '--random-agent'
[05:47:35] [WARNING] HTTP error codes detected during run:
403 (Forbidden) - 10 times, 429 (Too Many Requests) - 135 times
[05:47:35] [WARNING] your sqlmap version is outdated

[*] ending @ 05:47:35 /2026-03-24/

┌──(kali㉿kali)-[~/Downloads/Intern]
└─$
```

> Show:

- Terminal running SQLMap
- “not injectable” result

Result

- No SQL Injection vulnerability found
- SQLMap reported that parameters were not injectable

Analysis

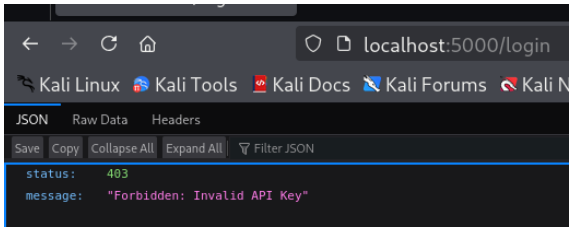
The application does not use a database or dynamic SQL queries. Instead, it performs direct value comparison:

```
if (username === "admin" && password === "1234")
```

Therefore, user input is not processed in a way that allows SQL injection.

Additional Observations

- Multiple **403 Forbidden** responses were observed due to missing/invalid API key



- Multiple **429 Too Many Requests** responses occurred due to rate limiting

These mechanisms effectively prevented automated attack attempts.

5. CSRF Testing

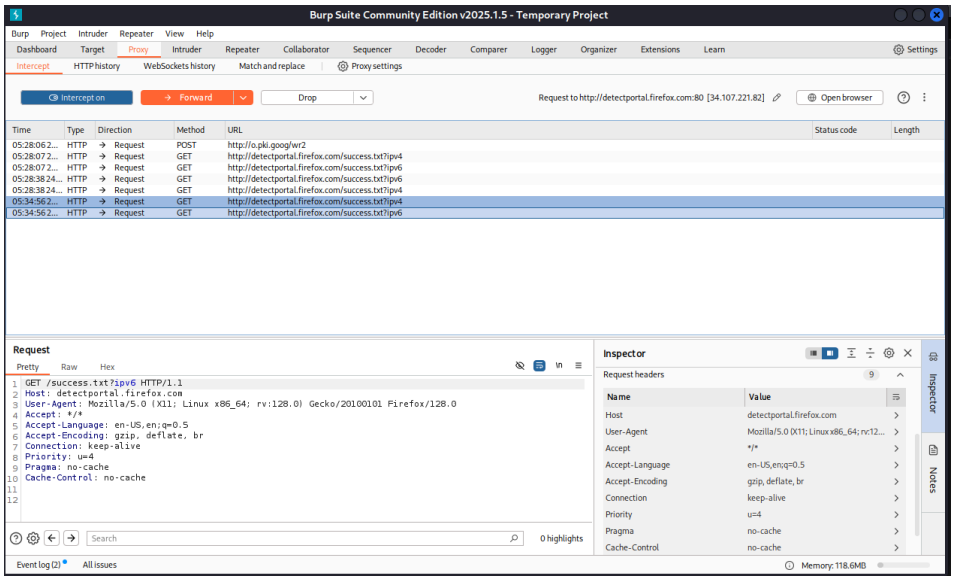
Methodology

CSRF testing was conducted using Burp Suite by intercepting and replaying HTTP requests.

Process

- Login request was captured using Burp Proxy
- Request was modified and replayed in Repeater

- Tested whether requests could be executed without proper validation

Results

- No CSRF vulnerability was identified in the current implementation
- The API requires a valid API key in headers, which prevents unauthorized request execution